OPERATING SYSTEM

COMPLETE GTU NOTES

All 9 Units | Detailed Answers | Algorithms | Diagrams | Examples

GTU Exam Preparation | B.E. Computer / IT Engineering

UNIT 1: INTRODUCTION TO OPERATING SYSTEMS (4 Hours)

1.1 Goals and Structure of an Operating System

An **Operating System (OS)** is system software that acts as an intermediary between users and computer hardware. It manages hardware resources and provides a convenient environment for running application programs.

Goals of OS:

- **Convenience:** Make computer use easy for the user.
- **Efficiency:** Use hardware resources (CPU, memory, I/O) efficiently.
- **Ability to evolve:** Allow development and testing of new system functions without interfering with existing ones.

Structure of OS:

```
User Programs / Applications
  → System Calls (API)
  → Operating System (Process Mgmt, Memory Mgmt, File System, I/O System)
  → Hardware (CPU, RAM, Disk)
```

1.2 Basic Functions of OS

- **Process Management:** Creating, scheduling, and terminating processes.
- **Memory Management:** Allocating and deallocating memory space as needed.
- **File System Management:** Organizing, storing, retrieving, naming, and protecting files.
- **I/O Device Management:** Managing I/O devices via device drivers.
- **Security and Protection:** Preventing unauthorized access to system resources.
- **Networking:** Managing network connections and data transfer.
- **Error Detection & Correction:** Detecting and handling hardware/software errors.

1.3 Interaction of OS with Hardware Architecture

The OS interacts with hardware through:

- **Interrupts:** Hardware sends interrupt signals to the CPU. The OS Interrupt Handler processes the interrupt and returns control to the user program.
- **Dual Mode Operation:** User Mode (limited access) and Kernel Mode (full access to all hardware).
- **System Calls:** Programs switch from user mode to kernel mode using system calls.
- **Direct Memory Access (DMA):** Allows I/O devices to access main memory independently of the CPU.

1.4 Types of Operating Systems

Type	Description	Example
Batch OS	Jobs grouped into batches; processed without user interaction.	IBM OS/360
Multiprogramming	Multiple programs in memory; CPU switches when one waits for I/O.	Unix

Multitasking	CPU rapidly switches between tasks; each process gets a time quantum.	Windows
Time Sharing	Multiple users share system simultaneously; each gets CPU time slice.	UNIX, Multics
Parallel OS	Multiple CPUs share memory; true simultaneous execution.	Multi-core systems
Distributed OS	Multiple networked computers appear as one system.	LOCUS, AMOEBA
Real-time OS	Guarantees response within strict time constraints.	VxWorks, FreeRTOS

1.5 System Calls

A **System Call** is a programming interface provided by the OS that allows user programs to request services from the kernel. It is the mechanism for transitioning from user mode to kernel mode.

- **Process Control:** fork(), exec(), exit(), wait()
- **File Management:** open(), close(), read(), write()
- **Device Management:** ioctl(), read(), write()
- **Information Maintenance:** getpid(), time(), getuid()
- **Communication:** pipe(), shmget(), msgget()

Note: When a system call is made, a trap is generated which switches the CPU to kernel mode, the OS performs the requested service, then returns to user mode.

UNIT 2: PROCESS AND THREADS MANAGEMENT (5 Hours)

2.1 Process Concept and Process States

A **Process** is a program in execution. It is an active entity (unlike a program which is a passive file on disk). A process includes: program code, program counter, stack, data section, and heap.

Process State Diagram (5-State Model):

```
New → Ready (admitted)
Ready → Running (scheduler dispatch)
Running → Waiting (I/O or event wait)
Waiting → Ready (I/O completion)
Running → Ready (interrupt / time quantum expired)
Running → Terminated (exit)
```

- **New:** Process is being created.
- **Ready:** Waiting to be assigned to a processor.
- **Running:** Being executed by CPU.
- **Waiting (Blocked):** Waiting for I/O or event.
- **Terminated:** Has finished execution.

2.2 Process Control Block (PCB)

The PCB is a data structure maintained by the OS for every process. It contains all information needed to manage a process.

- Process ID (PID)
- Process State
- Program Counter
- CPU Registers
- CPU Scheduling Info
- Memory Management Info
- Accounting Info
- I/O Status Info

2.3 Process vs Thread

Feature	Process	Thread
Address Space	Own address space	Shares parent address space
Overhead	More overhead to create	Less overhead to create
Context Switch	Slow	Fast
Communication	IPC (complex)	Shared memory (easy)
Failure Impact	Doesn't affect others	Can affect entire process

2.4 CPU Scheduling

Scheduling Criteria:

- **CPU Utilization:** Keep CPU as busy as possible.
- **Throughput:** Number of processes completed per unit time.
- **Turnaround Time:** Total time from submission to completion.
- **Waiting Time:** Total time spent in ready queue.
- **Response Time:** Time from submission to first response.

2.5 Scheduling Algorithms

A) FCFS (First Come First Served)

Processes are executed in the order they arrive. Non-preemptive. Simple but may cause convoy effect.

```
Example: P1(BT=24), P2(BT=3), P3(BT=3) arrive at time 0.
Gantt: P1(0-24) → P2(24-27) → P3(27-30)
Avg Waiting Time = (0 + 24 + 27) / 3 = 17 ms
Drawback: Convoy Effect – short processes wait behind long ones.
```

B) SJF (Shortest Job First)

Process with smallest burst time is selected next. Optimal for minimum average waiting time. Preemptive version = SRTF.

```
Example: P1(BT=6), P2(BT=8), P3(BT=7), P4(BT=3)
Order: P4(3) → P1(6) → P3(7) → P2(8)
Avg Waiting Time = (0 + 3 + 9 + 16) / 4 = 7 ms
```

C) Round Robin (RR)

Each process gets a fixed time quantum (q). After quantum expires, it goes back to the ready queue. Best for time-sharing systems.

```
Example: P1(BT=24), P2(BT=3), P3(BT=3), q=4
Schedule: P1(0-4) → P2(4-7) → P3(7-10) → P1(10-14) → ... → P1(26-30)
Note: Too small q → many context switches; too large q → degenerates to FCFS.
```

D) Priority Scheduling

Each process is assigned a priority; CPU allocated to highest priority process. Can be preemptive or non-preemptive.

- **Problem:** Starvation — low priority process never executes.
- **Solution:** Aging — gradually increase priority of waiting processes.

2.6 System Calls: fork(), exec(), wait(), ps

```
fork()  – Creates child process as copy of parent. Returns 0 to child, PID to parent.
exec()  – Replaces current process image with new program.
wait()  – Parent waits for child process to complete.
ps      – Displays currently running processes.

Example:
  int pid = fork();
```

```
if (pid == 0) {
    exec("/bin/ls", ...);  // Child executes ls
} else {
    wait(NULL);            // Parent waits
    printf("Child done\n");
}
```

UNIT 3: CONCURRENCY AND SYNCHRONIZATION (5 Hours)

3.1 Principles of Concurrency

Concurrency refers to the execution of multiple processes or threads simultaneously or in overlapping time periods. Problems arising from concurrency:

- **Race Condition:** Multiple processes access shared data simultaneously; outcome depends on execution order.
- **Deadlock:** Processes block each other waiting for resources.
- **Starvation:** A process is perpetually denied resources.
- **Non-determinism:** Same program may produce different results each run.

3.2 Critical Section Problem

A **Critical Section** is a segment of code where a process accesses shared resources. Requirements for a valid solution:

- **Mutual Exclusion:** Only one process can be in CS at a time.
- **Progress:** If no process is in CS, a process wanting to enter should be able to.
- **Bounded Waiting:** A process must not wait indefinitely to enter CS.

3.3 Mutual Exclusion

Software Approaches:

- **Strict Alternation:** Processes take turns. Violates progress condition.
- **Peterson's Solution:** Uses two variables (turn, flag[]) for two processes. Satisfies all three conditions.
- **Bakery Algorithm:** Works for N processes. Each takes a number; lowest number enters CS.

Hardware Approaches:

- **Disable Interrupts:** Works on uniprocessor but not multiprocessor.
- **Test-and-Set:** Atomic instruction. If lock is false, set to true and enter CS.
- **Compare-and-Swap:** Atomic: if value equals expected, replace with new value.

3.4 Semaphores

A **Semaphore** is a synchronization tool — an integer variable accessed only through two atomic operations:

```
wait(S):    while(S <= 0);   // busy wait
            S--;

signal(S):  S++;
```

- **Binary Semaphore (Mutex):** Value is 0 or 1. Used for mutual exclusion.
- **Counting Semaphore:** Value ranges over unrestricted domain. Used for resource counting.

```
Usage for Mutual Exclusion:
  Semaphore S = 1;
```

```
Process Pi:
  wait(S);
    // Critical Section
  signal(S);
```

3.5 Short Notes

Pipes:

A pipe is a unidirectional communication channel between two processes. Ordinary pipes: only between parent-child. Named pipes (FIFOs): between unrelated processes.

Message Passing:

Processes communicate by send(message) and receive(message) primitives without sharing memory. Can be direct or indirect, synchronous or asynchronous.

Signals:

Software interrupts sent to a process to notify it of an event. Examples: SIGKILL (terminate), SIGSTOP (pause), SIGSEGV (segmentation fault).

Monitors:

A high-level synchronization construct. Encapsulates shared variables and procedures. Only one process can be active inside a monitor at a time. Uses condition variables (wait, signal).

UNIT 4: INTER-PROCESS COMMUNICATION (6 Hours)

4.1 Race Condition and Critical Section

A **Race Condition** occurs when two or more processes read and write shared data concurrently, and the final outcome depends on the exact timing of their execution.

Example: Two processes both execute "counter++" concurrently. Internally: (1) read counter, (2) increment, (3) write back. If interleaved, one update can be lost.

4.2 Peterson's Solution

```
int turn;
boolean flag[2] = {false, false};

Process P0:              Process P1:
  flag[0] = true;          flag[1] = true;
  turn = 1;                turn = 0;
  while(flag[1]&&turn==1); while(flag[0]&&turn==0);
    // Critical Section      // Critical Section
  flag[0] = false;         flag[1] = false;
```

Peterson's solution satisfies Mutual Exclusion, Progress, and Bounded Waiting for two processes.

4.3 Producer-Consumer Problem (with Semaphore)

```
Semaphore mutex = 1;   // mutual exclusion
Semaphore empty = N;   // N empty slots initially
Semaphore full  = 0;   // 0 full slots initially

PRODUCER:              CONSUMER:
  wait(empty);           wait(full);
  wait(mutex);           wait(mutex);
  // add to buffer       // remove from buffer
  signal(mutex);         signal(mutex);
  signal(full);          signal(empty);
```

4.4 Classical IPC Problems

A) Readers-Writers Problem

Multiple readers can read simultaneously. A writer needs exclusive access.

```
WRITER:                  READER:
  wait(wrt);               wait(mutex);
    // write               readcount++;
  signal(wrt);             if(readcount==1) wait(wrt);
                           signal(mutex);
                             // read
                           wait(mutex);
                           readcount--;
```

```
if(readcount==0) signal(wrt);
signal(mutex);
```

B) Dining Philosophers Problem

Five philosophers at a round table; one chopstick between each pair. A philosopher needs both chopsticks to eat. Solutions to prevent deadlock:

- Allow only 4 philosophers to sit simultaneously.
- Pick up both chopsticks atomically (with mutex).
- Odd philosophers pick left first; even pick right first.

4.5 Message Passing vs Shared Memory

Feature	Message Passing	Shared Memory
Mechanism	send/receive primitives	Read/Write shared address
Data Size	Useful for small data	Faster for large data
Sync	OS manages communication	Processes manage sync
Network	Works across networks	Faster on single machine
Examples	Pipes, Sockets, MPI	shmget(), shmat() in Unix

4.6 Event Counters

An **Event Counter** is a non-negative integer that is only ever incremented. Operations: Read(ec), Advance(ec) (increment), Await(ec, v) (wait until ec >= v). Used to coordinate producer-consumer without risk of deadlock.

UNIT 5: DEADLOCK (3 Hours)

5.1 Four Necessary Conditions for Deadlock

All four must hold simultaneously for deadlock to occur:

- **1. Mutual Exclusion:** At least one resource held in non-shareable mode.
- **2. Hold and Wait:** A process holds at least one resource and waits for more.
- **3. No Preemption:** Resources cannot be forcefully taken; must be released voluntarily.
- **4. Circular Wait:** A set {P0..Pn} where each Pi waits for resource held by P(i+1), and Pn waits for P0.

5.2 Deadlock vs Starvation vs Livelock

Feature	Deadlock	Starvation	Livelock
Status	Permanently blocked	Waits indefinitely	Keeps changing state
Progress	No process proceeds	Others proceed	No real progress
Example	P1 waits for R2 (held by P2), P2 waits for R1	Low-priority process never gets CPU	Two processes keep yielding to each other

5.3 Resource Allocation Graph (RAG)

- Processes: circles (P1, P2); Resources: rectangles with dots
- **Request Edge:** Pi → Rj (Pi requests Rj)
- **Assignment Edge:** Rj → Pi (Rj assigned to Pi)
- No cycle → No deadlock; Cycle with single instance → Deadlock; Cycle with multiple instances → May or may not be deadlock

5.4 Handling Deadlock

A) Deadlock Prevention

- **Eliminate Mutual Exclusion:** Make resources shareable (not always possible).
- **Eliminate Hold and Wait:** Request all resources before start, or release all before new request.
- **Allow Preemption:** Preempt resources from another process if needed.
- **Eliminate Circular Wait:** Impose total ordering of resource types; request in order.

B) Deadlock Avoidance – Banker's Algorithm

OS checks before granting resources if the system remains in a safe state. A state is **safe** if there exists a safe sequence where all processes can complete.

```
Data Structures (n processes, m resources):
  Available[m]       – available instances of each resource
  Max[n][m]          – maximum demand of each process
  Allocation[n][m]   – currently allocated
  Need[n][m]         = Max - Allocation

Safety Algorithm:
```

1. Work = Available; Finish[i] = false for all i
2. Find i such that Finish[i]=false AND Need[i] <= Work
3. Work = Work + Allocation[i]; Finish[i] = true
4. Repeat step 2. If all Finish[i]=true → SAFE; else UNSAFE

C) Deadlock Detection & Recovery

• **Detection:** Single instance: use RAG, look for cycles. Multiple instances: use detection algorithm similar to Banker's.

• **Recovery – Process Termination:** Abort all deadlocked processes, or abort one at a time.

• **Recovery – Resource Preemption:** Select a victim, preempt its resources, rollback to safe state.

UNIT 6: MEMORY MANAGEMENT (7 Hours) — VERY IMPORTANT ★

6.1 Memory Management Requirements

- **Relocation:** Program can be loaded anywhere in memory.
- **Protection:** Processes cannot access each other's memory.
- **Sharing:** Multiple processes may share same memory region.
- **Logical Organization:** Programs structured as modules.
- **Physical Organization:** Moving information between main memory and disk.

6.2 Fixed vs Variable Partitioning

Feature	Fixed Partitioning	Variable Partitioning
Division	Fixed-size partitions	Dynamic partitions
Fragmentation	Internal (wasted inside partition)	External (wasted between partitions)
Flexibility	Simple, limited processes	More flexible, better utilization

6.3 Memory Allocation Strategies

Given: Free blocks of 100KB, 500KB, 200KB, 300KB, 600KB. Process needs 212KB.

- **First Fit:** Allocate first block large enough → 500KB block. Fast but may waste space at beginning.
- **Best Fit:** Allocate smallest sufficient block → 300KB block. Minimizes wasted space but slow.
- **Worst Fit:** Allocate largest available block → 600KB block. Leaves largest leftover for future.

Note: First Fit and Best Fit generally better than Worst Fit. First Fit is fastest.

6.4 Fragmentation

- **Internal Fragmentation:** Allocated memory larger than requested; wasted space inside partition. Occurs in fixed partitioning and paging.
- **External Fragmentation:** Enough total memory exists but not contiguous. Occurs in variable partitioning.
- **Solution:** Compaction (shuffle memory to consolidate free space) or Paging/Segmentation.

6.5 Swapping

Swapping moves a process from main memory to secondary storage (disk) to free up memory for other processes. When the process needs to run again, it is swapped back in. **Roll-out/roll-in** is used for priority-based scheduling.

6.6 Paging

Paging divides physical memory into fixed-size **frames** and logical memory into **pages** of the same size. Eliminates external fragmentation.

```
Logical Address = [Page Number | Offset]
                        |
```

```
                Page Table Lookup
                        |
Physical Address = [Frame Number | Offset]

Example: Page size = 4KB, Logical address = 13056
  Page number = 13056 / 4096 = 3
  Offset      = 13056 mod 4096 = 768
  If page 3 → frame 6: Physical address = 6 x 4096 + 768 = 25344
```

6.7 Demand Paging and Page Fault

Demand Paging: Pages loaded into memory only when needed, not all at once. Saves memory.

Page Fault Handling Steps:

- OS checks if address is valid. If not → abort.
- Find a free frame. If none → run page replacement algorithm.
- Load the required page from disk into the free frame.
- Update page table, mark page as valid.
- Restart the instruction that caused the page fault.

6.8 Virtual Memory

Virtual memory allows execution of processes not completely in memory. It separates logical memory from physical memory, allowing the logical address space to be much larger than physical memory.

6.9 Page Replacement Algorithms ★

A) FIFO (First In First Out)

Replace the page that has been in memory the longest. Suffers from **Belady's anomaly**.

```
Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2  |  Frames=3
Ref:  7  0  1  2  0  3  0  4  2  3  0  3  2
F1:   7  7  7  2  2  2  2  4  4  4  0  0  0
F2:   -  0  0  0  0  0  0  0  2  2  2  2  2
F3:   -  -  1  1  1  3  3  3  3  3  3  3  3
Flt:  F  F  F  F  -  F  -  F  F  F  F  -  -
Total Page Faults = 9
```

B) LRU (Least Recently Used)

Replace the page not used for the longest time. No Belady's anomaly.

```
Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2  |  Frames=3
Ref:  7  0  1  2  0  3  0  4  2  3  0  3  2
F1:   7  7  7  2  2  2  2  4  4  4  0  0  0
F2:   -  0  0  0  0  0  0  0  0  3  3  3  3
F3:   -  -  1  1  1  3  3  3  2  2  2  2  2
Flt:  F  F  F  F  -  F  -  F  F  F  F  -  -
Total Page Faults = 8
```

C) Optimal Page Replacement

Replace the page that will not be used for the longest time in the future. Theoretically optimal (minimum faults) but not implementable in practice. Used as a benchmark.

```
Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2  |  Frames=3
Total Page Faults = 5 (best possible)
```

6.10 Thrashing

Thrashing occurs when a process spends more time paging than executing. The CPU is busy servicing page faults rather than running productive code.

- **Cause:** Too many processes competing for too few frames.

Prevention:

- **Working Set Model:** Keep in memory only pages in process's recent working set.
- **Page Fault Frequency (PFF):** If fault rate too high → add frames; too low → remove frames.
- Reduce degree of multiprogramming.

6.11 Segmentation

Segmentation divides logical memory into **variable-length segments** corresponding to logical units (main program, function, stack, data). A **Segment Table** maps segment number to base address and limit. Supports sharing and protection but suffers from external fragmentation.

UNIT 7: I/O MANAGEMENT AND DISK SCHEDULING (5 Hours)

7.1 I/O Devices and Organization

- **Block Devices:** Store info in fixed-size blocks (disk, USB). Can seek to any block.
- **Character Devices:** Deliver stream of characters (keyboard, mouse, serial port).

Layered I/O Organization: User-level I/O → Device-independent OS software → Device drivers → Interrupt handlers → Hardware

7.2 I/O Buffering

- **Single Buffer:** One buffer. Producer puts in, consumer takes out.
- **Double Buffer:** Two buffers. While consumer empties one, producer fills other. Reduces idle time.
- **Circular Buffer:** Multiple buffers in ring. Better for bursty I/O.

7.3 Disk Scheduling Algorithms ★

Request queue: 98, 183, 37, 122, 14, 124, 65, 67 | Initial head: 53

A) FCFS

Service requests in order of arrival. Fair but not efficient.

```
Order: 53→98→183→37→122→14→124→65→67
Total movement = 640 cylinders
```

B) SSTF (Shortest Seek Time First)

Service the request closest to current head position. May cause starvation of far requests.

```
Order: 53→65→67→37→14→98→122→124→183
Total movement = 236 cylinders
```

C) SCAN (Elevator Algorithm)

Head moves in one direction servicing requests until disk end, then reverses.

```
Order (moving high): 53→65→67→98→122→124→183→199→37→14
Total movement = 208 cylinders
```

D) C-SCAN (Circular SCAN)

Like SCAN but when reaching disk end, head jumps back to beginning and services in one direction only. Provides more uniform wait time.

```
Order: 53→65→67→98→122→124→183→199→0→14→37
Total movement = 382 cylinders (includes jump from 199 to 0)
```

7.4 RAID

- **RAID 0:** Striping — speed, no redundancy.
- **RAID 1:** Mirroring — full redundancy.
- **RAID 5:** Block-level striping with distributed parity — balance of performance and reliability.

7.5 Disk Cache

A buffer in main memory for disk sectors. When a sector is read, it is stored in cache. Subsequent reads are served from cache, avoiding disk access. Replacement policy: LRU or LFU.

UNIT 8: UNIX/LINUX OPERATING SYSTEM (5 Hours)

8.1 Architecture of Unix/Linux

```
User Applications
  Shell (bash / sh / zsh) — Command-line interpreter
    System Call Interface
      Kernel (Process Mgmt, Memory Mgmt, File System, I/O, Network)
        Hardware
```

8.2 Role and Functions of the Kernel

- **Process Management:** Creates, schedules, terminates processes using CFS scheduler.
- **Memory Management:** Manages virtual memory, paging, memory allocation (kmalloc).
- **File System:** Provides Virtual File System (VFS) abstraction. Supports ext4, XFS, NTFS, FAT.
- **Device Drivers:** Manages interaction with hardware devices.
- **Networking:** TCP/IP stack implementation.
- **Security:** User permissions, SELinux, capabilities.

8.3 Important System Calls

```
Process:          File:                  IPC:
  fork()            open(path, flags)      pipe()
  exec()            read(fd, buf, n)       socket()
  exit()            write(fd, buf, n)      shmget()
  wait()            close(fd)

Information:
  getpid()  getuid()
```

8.4 Linux Directory Structure

```
/               Root directory
■■■ /bin         Essential user commands (ls, cp, mv)
■■■ /sbin        System admin commands (fsck, ifconfig)
■■■ /etc         Configuration files
■■■ /home        User home directories
■■■ /usr         User programs and data
■   ■■■ /usr/bin    Non-essential user programs
■   ■■■ /usr/lib    Libraries
■■■ /var         Variable data (logs, mail)
■■■ /tmp         Temporary files (cleared on reboot)
■■■ /dev         Device files
■■■ /proc        Virtual filesystem for process info
■■■ /lib         Shared libraries
■■■ /mnt         Mount points for external filesystems
```

8.5 System Administration Commands

```
Process:        ps aux | top | kill PID | nice | renice
User:           useradd | passwd | usermod | userdel
Filesystem:     mount | df -h | du -sh | chmod | chown | fsck
Network:        ifconfig | netstat | ping | traceroute | iptables
```

8.6 Linux vs Windows Comparison

Feature	Linux	Windows
Source Code	Open source (GPL)	Proprietary (closed)
File System	ext4, XFS, Btrfs	NTFS, FAT32, exFAT
Process Scheduling	CFS (Completely Fair)	Priority-based preemptive
Security Model	DAC + SELinux/AppArmor	ACL + UAC
CLI	Bash/Shell (powerful)	CMD / PowerShell
Cost	Free	Paid license
Usage	Servers, embedded, supercomputers	Desktops, gaming, enterprise

UNIT 9: VIRTUALIZATION CONCEPTS

9.1 What is Virtualization?

Virtualization is the creation of a virtual version of hardware, storage devices, and network resources. It allows a single physical machine to run multiple operating systems simultaneously. A **Hypervisor (VMM)** creates and manages Virtual Machines (VMs).

9.2 True/Pure Virtualization (Full Virtualization)

The guest OS runs without modification — it doesn't know it's inside a VM. The hypervisor intercepts and emulates all privileged instructions.

```
Guest OS 1 | Guest OS 2 | Guest OS 3
      Hypervisor / VMM
         Physical Hardware
```

9.3 Approaches to Virtualization

Type 1 (Bare Metal) Hypervisor:

Runs directly on physical hardware without a host OS. More efficient and secure. Examples: VMware ESXi, Microsoft Hyper-V, Xen.

Type 2 (Hosted) Hypervisor:

Runs on top of a host OS. Easier to install and use. Examples: VMware Workstation, Oracle VirtualBox.

Paravirtualization:

Guest OS is modified to be aware of virtualization. Uses hypercalls instead of privileged instructions. Better performance than full virtualization. Example: Xen with modified guest kernel.

- **Processor Virtualization:** Managing privileged instructions, context switching between VMs via trap-and-emulate.
- **Memory Virtualization:** Each VM has its own virtual address space. Hypervisor manages shadow page tables or uses Intel EPT/AMD NPT.
- **I/O Virtualization:** Virtual device drivers, shared access to physical devices.

9.4 Short Notes on Hypervisors and VMs

Product	Type	Key Features
VMware ESXi	Type 1	Enterprise-grade; used in data centers; high performance.
Microsoft Hyper-V	Type 1	Built into Windows Server; supports Windows and Linux guests.
Xen	Type 1	Open-source; powers Amazon EC2; supports full and paravirtualization.
Java VM (JVM)	Language VM	Runs Java bytecode; write-once-run-anywhere; JIT compilation.
Android ART	Language VM	Replaced Dalvik; AOT compilation; faster runtime performance.

GTU EXAM PREPARATION TIPS

High Priority Topics (Most Asked in GTU):

- Page replacement algorithms (FIFO, LRU, Optimal) with full numerical examples — Unit 6.
- Paging mechanism, demand paging, virtual memory, thrashing — Unit 6.
- CPU scheduling algorithms (FCFS, SJF, RR, Priority) with Gantt charts — Unit 2.
- Banker's Algorithm numerical (deadlock avoidance) — Unit 5.
- Producer-Consumer, Dining Philosophers, Readers-Writers with semaphore code — Unit 4.
- Disk scheduling (FCFS, SSTF, SCAN, C-SCAN) with cylinder movement calculation — Unit 7.

Important Diagrams to Practice:

- Process State Diagram (5-state model)
- PCB structure
- Paging address translation mechanism
- Deadlock Resource Allocation Graph
- Unix/Linux layered architecture
- Disk scheduling head movement

Reference Books:

- Silberschatz, Galvin, Gagne — *Operating System Concepts*
- Andrew Tanenbaum — *Modern Operating Systems*
- William Stallings — *Operating Systems: Internals and Design Principles*
- Dhamdhere — *Operating Systems: A Concept-Based Approach*

Best of luck for your GTU exams!